

SixOS

Introduction

6COM Engineering Team

October 8, 2026

Contents

1	SixOS — A Web-Native, AI-First Operating System	3
1.1	The Vision	3
1.2	The Core Idea	3
1.3	Web-Native: The Web Is the Platform	3
1.4	AI-First: Intelligence as an OS Service	4
1.5	Why WebAssembly?	5
1.6	Why Rust for the OS Core	5
1.7	Design Pillars	5
1.8	Target Platform	6
2	SixOS — Architecture at a Glance	6
2.1	The System Map	7
2.2	No Context Switching	7
2.3	The WASM Runtime	7
2.4	Capability-Based Security	8
2.5	Multi-Core Design	8
2.6	Real-Time Scheduling	9
2.7	Inter-Process Communication	9
3	SixOS — Userspace and Applications	9
3.1	Supported Languages (Applications)	10
3.2	System Services (Native Rust)	10
3.2.1	init — Service Manager	11
3.2.2	netd — Networking Service	11
3.2.3	gpud — Graphics Service	11
3.2.4	hidd — Input Service	11
3.2.5	appmgrd — Application Manager	11
3.2.6	shelld — Desktop Shell	11
3.3	The Browser Application	12
3.4	Filesystem	12
3.5	Two-Layer POSIX Compatibility	12
3.6	The Application Model	13
4	SixOS — Supported Devices Plan	13
4.1	Tier 1 — Reference Development Platform (Current)	13
4.2	Tier 2 — Single-Board Computers	13
4.3	Tier 3 — Handheld	14
4.4	Tier 4 — IoT and Edge	14
4.5	Tier 5 — UAV and Aerial Robotics	14
4.6	Tier 6 — Automotive IVI	15
4.7	What Each Tier Shares	16
5	SixOS — Current Status	17

5.1	Kernel — Working	18
5.2	WASM Runtime — Working	18
5.3	System Services — Working	19
5.4	Applications — Working	19
5.5	Hardware Bring-Up — In Progress	19
6	SixOS — The Real-Time Variant	19
6.1	Why a No-WASM Variant	19
6.2	What Stays the Same	20
6.3	What Changes	20
6.4	Where Each Variant Runs	21
6.5	Why This Matters	21
7	Appendix A — Capability-Based Security in Context	22
7.1	The Question	22
7.2	The Alternatives	22
7.3	Capability-Based Systems	23
7.4	Worked Example: The Same Afternoon, Under Five Models . . .	23
7.5	Side-by-Side	24
7.6	Why This Decision Is Load-Bearing	25

1 SixOS — A Web-Native, AI-First Operating System

Project started: June 2026.

1.1 The Vision

Computing has converged on two platforms: the web, where applications are links and interfaces are documents; and AI, where intent replaces instruction. Every mainstream operating system treats both as guests — a browser app running on top, an assistant bolted on the side. SixOS is designed from first principles for the world they created:

SixOS is a web-native, AI-first operating system. The web platform is the application model — not a guest inside it. AI is an OS service — not a feature of one app. Everything else follows from those two decisions.

1.2 The Core Idea

Under the vision sits a simple technical premise: **applications run WebAssembly; the OS core runs Rust.**

No ELF binaries. No dynamic linking. No legacy ABIs. Every user application is a `.wasm` module running in a sandboxed interpreter. The OS core — the kernel plus system services like init, networking, graphics, input, and the window manager — is native Rust, compiled with the kernel. Applications reach the OS through exactly one interface: a set of host functions that WASM modules can call. That's it.

This simplicity yields something unusual: an OS that is secure by construction, deterministic under load, and language-agnostic in its application layer.

1.3 Web-Native: The Web Is the Platform

On SixOS, “web app” and “app” are the same thing. There is no separate native SDK, no packaging format, no app store pipeline — because the web platform already solved distribution:

- **URLs are the app model.** An application is something you can link to. Installing an app means pinning a URL; updating it means the publisher updates the site. Versioning, delta updates, and offline caching are properties of the platform, not of a store.
- **HTML, CSS, and JavaScript render through the system browser engine.** The browser is not an application on SixOS — it is a system

service, like the window manager. Any window can host web content; any app can embed it.

- **Performance code drops to WebAssembly.** The same sandbox that runs a full page’s logic runs a codec, a physics engine, or a spreadsheet recalculation at near-native speed — no plugin, no escape hatch.
- **WebGPU is the graphics API.** The system’s GPU service already speaks a WebGPU-native API, so accelerated rendering looks identical from a web page and from any other application.
- **One security model end to end.** Web pages and WASM apps run in the same sandbox, speak the same host-function ABI, and hold the same capability tokens. The gap between “web security” and “OS security” — the gap every browser sandbox dances around — does not exist.

The result: developers build for SixOS exactly the way they build for the web, because it *is* the web. The 30-year investment in web tooling, frameworks, and skills transfers directly.

1.4 AI-First: Intelligence as an OS Service

SixOS treats AI the way it treats graphics or networking — a capability of the system, available to every application through the same host-function interface, governed by the same capability security:

- **A system-wide assistant with real authority boundaries.** The assistant can act on the user’s behalf — open a window, compose a message, summarize a document — but only through capabilities the user has granted it, revocable at any time. An AI with a capability token is just another process: least-privilege by construction, not by policy document.
- **On-device inference.** Language and vision models run locally on the RISC-V vector unit, not in a datacenter. Latency is bounded, prompts never leave the machine, and the assistant keeps working with the network down. The real-time scheduler guarantees inference doesn’t starve interactive workloads — and vice versa.
- **Natural language as a first-class input modality.** Alongside keyboard, mouse, and touch, the input daemon speaks intent. “Dim the screen and summarize my last three documents” compiles to the same capability-gated operations any program would invoke.
- **Agents are sandboxed WASM too.** An AI agent someone else wrote is a .wasm module like any other application. It cannot do anything the user’s own code couldn’t — the sandbox and capability system apply identically to human and machine authors.
- **AI-aware system services.** Search is semantic. The shell predicts, the window manager understands layout context, and accessibility is narration by default rather than an afterthought.

Because the OS core is a small, auditable Rust TCB and every application — human or AI-authored — is a sandboxed module, SixOS can safely hand agency to software. That is what “AI-first” means in practice: not more chatbots, but an OS whose security model was designed for a world where programs act on people’s behalf.

1.5 Why WebAssembly?

WebAssembly was designed as a browser sandbox, but its properties make it an ideal universal application ABI:

- **Sandboxed by construction:** WASM modules cannot escape their linear memory. There is no way for an application to forge a pointer, jump to arbitrary code, or corrupt kernel memory. The isolation is built into the execution model, not bolted on via page tables.
- **Single ABI surface:** Instead of supporting ELF, COFF, dynamic linking, and multiple syscall ABIs, SixOS exposes one interface — host functions. Every language that compiles to WASM speaks the same language.
- **Language-agnostic:** Rust, Go, C, C++, Zig, C#, Swift, Kotlin, D, Nim, Haskell, AssemblyScript, and more — anything that targets WASI Preview 1 runs on SixOS. Developers choose their language; the OS doesn’t care.
- **Deterministic:** WASM execution is well-defined. No undefined behavior, no architecture-specific quirks, no JIT compilation pauses.

1.6 Why Rust for the OS Core

The kernel and all core system services are written in safe Rust. This eliminates entire categories of vulnerabilities at compile time:

- No use-after-free (ownership system)
- No buffer overflows (bounds checking)
- No data races (borrow checker)
- No null pointer dereferences (Option instead of null)

Unsafe blocks are audited, bounded, and kept to a minimum — primarily for RISC-V register access and MMIO operations. For an OS that intends to host AI agents, the size and audibility of the trusted computing base is not an implementation detail — it is the product.

1.7 Design Pillars

SixOS is built on five pillars:

Security: Capability-based access control throughout. No process has “root” access. Every kernel operation requires an unforgeable token (capability) that carries specific rights. Capabilities can be minted, derived, transferred, and revoked — and an AI agent’s permissions are just capabilities, no more and no less.

Web-Native: One application platform — the web. URLs for distribution, the system browser engine for UI, WebAssembly for compute, WebGPU for graphics. No second native platform to build, secure, and maintain.

AI-First: Intelligence is a system service: on-device inference, natural-language input, and an assistant whose authority is bounded by the same capability tokens that govern every other process.

Real-Time Determinism: A priority-preemptive scheduler with bounded interrupt latency under 50 microseconds. Real-time threads can be pinned to specific cores. Priority inversion is prevented by the priority ceiling protocol. Determinism is what makes an OS safe to hand agency to.

Simplicity: A single application ABI (WASM host functions), a single address space (kernel identity-mapped), a single application executable format (.wasm). Core services ship as part of the OS image — no separate packaging, dynamic linking, or legacy compatibility layers to carry forward.

1.8 Target Platform

SixOS targets RISC-V 64-bit (RV64) machines — an open instruction set whose vector extensions are built for the matrix math that on-device AI demands. Development and testing run on QEMU’s virt machine with VirtIO devices (block, net, GPU, input, RNG). Support for physical RISC-V hardware follows the same device interface.

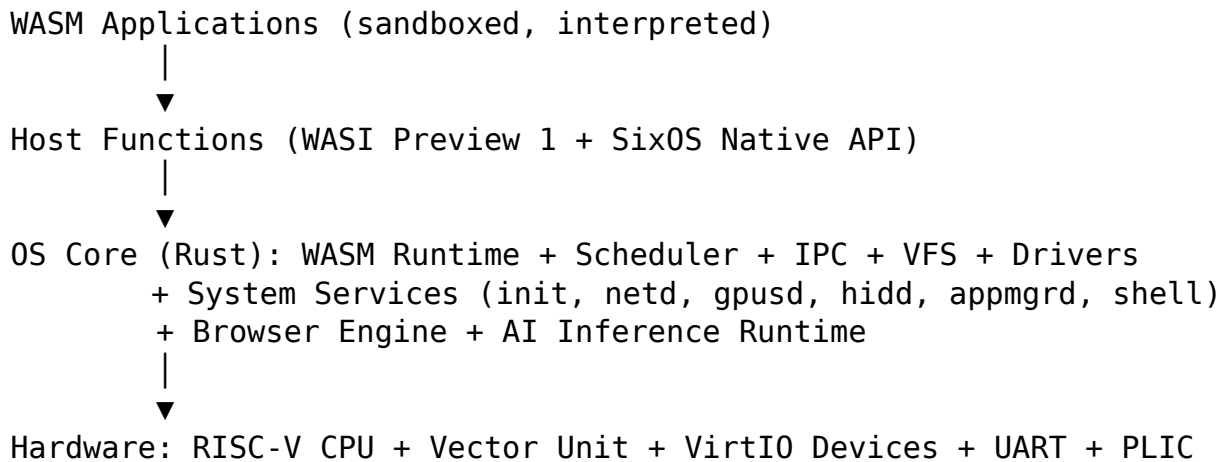
2 SixOS — Architecture at a Glance

Project started: June 2026.

SixOS is a web-native, AI-first operating system: the web platform is the application model, and AI is a system service available to every app. Under that vision, it replaces the traditional kernel/userspace boundary with something simpler: a WASM runtime that hosts all application code, running inside the kernel’s address space, alongside the native Rust system services — including the system browser engine and the on-device AI inference runtime.

2.1 The System Map

At the highest level, SixOS has four layers:



There is no “userspace” in the traditional sense — no separate address spaces, no TLB flips on context switches, no privilege mode transitions for syscalls. Applications are WASM modules interpreted inside the kernel’s address space; core services are native Rust code built into the same image.

2.2 No Context Switching

In a traditional microkernel, every syscall requires a message to a userspace server, which means two context switches and two TLB flushes. In SixOS, WASM modules run in the kernel’s address space with sandboxed linear memory. When a WASM module calls a host function (say, `fd_write`), it is a simple function call — no privilege mode change, no address space switch, no TLB flush. Core services, being native Rust in the same address space, are ordinary function calls away — no IPC marshalling needed on the hot path.

The isolation comes from the WASM execution model itself, not from hardware page tables. Each WASM instance has its own linear memory, and the interpreter enforces bounds checking on every load and store. This replaces hardware page table isolation with software-enforced sandboxing that is both faster and more flexible.

2.3 The WASM Runtime

The kernel embeds a minimal WASM bytecode interpreter:

- **Module loading:** `.wasm` files are parsed from standard WASM binary format — sections for types, imports, functions, exports, memory, globals, code, and data.

- **Bytecode execution:** The interpreter decodes and executes WASM instructions directly. No JIT, no ahead-of-time compilation. The interpreter is around 1,500 lines of Rust and supports the full WASM instruction set.
- **Module cache:** Loaded modules are cached for reuse. Multiple instances can be spawned from the same module without re-parsing.
- **Host function dispatch:** When WASM code calls an imported function (like `fd_write`), the interpreter routes it to the kernel’s registered handler through a dispatch table. The handler runs natively in the kernel and returns results to the WASM caller.

2.4 Capability-Based Security

Every kernel object — a memory region, an IPC channel, a file handle, a device — is referenced by an unforgeable capability. A capability is a token that identifies an object and carries a set of rights (read, write, execute, grant, transfer, etc.).

Key properties:

- **No ambient authority:** A process can only access objects for which it holds a capability. There is no “root user”, no global namespace, no implicit trust.
- **Minting:** A capability holder with the “grant” right can create a new capability for the same object with a subset of rights.
- **Derivation:** Similar to minting but preserves the object type.
- **Transfer:** Capabilities can be sent between processes via IPC channels.
- **Revocation:** A capability can be revoked, which recursively revokes all descendants in the derivation tree.

2.5 Multi-Core Design

SixOS supports 1-8 cores (HARTs in RISC-V terminology) with a shared-memory symmetric multiprocessing model:

- **Per-CPU runqueues:** Each core has its own scheduler runqueue, avoiding lock contention on the common path.
- **Work-stealing:** When a core’s runqueue is empty, it can steal work from another core’s queue.
- **Partitioned real-time:** Real-time threads can be pinned to specific cores, guaranteeing isolation from best-effort workloads.
- **Per-CPU timers:** Each core has its own timer interrupt for independent scheduling ticks.

2.6 Real-Time Scheduling

The scheduler uses a combination of fixed-priority and EDF (Earliest Deadline First) scheduling:

- **Priority levels 0-99:** Best-effort (background through interactive)
- **Priority levels 100-199:** Real-time soft deadline
- **Priority levels 200-255:** Kernel and real-time hard deadline

Guarantees: - Interrupt latency: under 50 microseconds - Context switch: under 10 microseconds - Host function call: under 5 microseconds - Priority inversion: prevented via the priority ceiling protocol - Lock-free read paths for scheduler runqueue lookups

2.7 Inter-Process Communication

IPC in SixOS is capability-gated message passing over channels:

- **Channels:** A pair of endpoints, each owned by a process. Messages are typed byte buffers with optional capability attachments.
- **Ring buffers:** Each channel endpoint has a bounded ring buffer (64 messages of up to 512 bytes each). Producers and consumers operate without heap allocation.
- **Blocking and non-blocking:** Senders and receivers can choose blocking or non-blocking operations. Blocking calls suspend the coroutine, not the kernel thread.
- **Notifications:** Lightweight signal objects for event delivery without message payloads.
- **Shared memory:** Large data transfers use shared memory regions mapped into both processes' linear memories, avoiding copy overhead.

3 SixOS — Userspace and Applications

SixOS is web-native and AI-first: the browser engine is a system service, and AI inference is one of the host functions any application can call. It draws a clear line between the two kinds of software it runs:

- **Applications** are WebAssembly-only. Every user application is a .wasm module compiled from Rust, Go, C, or any language that targets WASI Preview 1. They run sandboxed in the WASM interpreter and reach the OS exclusively through host functions.
- **Core system services** are native Rust, built into the OS image with the kernel. They are trusted components — not sandboxed — and provide networking, graphics, input, window management, and the desktop shell.

3.1 Supported Languages (Applications)

Language	Target	Toolchain
Rust	wasm32-wasip1	rustc with standard library support
Go	G00S=wasip1 GOARCH=wasm	go build (Go 1.21+)
C / C++ C++ (CMake)	wasm32-wasi wasm32-wasi	clang with WASI SDK cmake with WASI SDK toolchain file
Zig	wasm32-wasi	zig build
AssemblyScript	wasm32 + WASI	asc (TypeScript subset)
C# / .NET	wasm32-wasi	dotnet publish with WASI SDK
Kotlin	wasm-wasi	Kotlin/Wasm with @kotlinux-wasi
Swift	wasm32-unknown-wasi	SwiftWasm toolchain
D	wasm32-unknown-wasi	ldc2 with -mtriple=wasm32-unknown-wasi
Nim	wasm32	nim with --cpu:wasm32 and wasi backend
Haskell	wasm32-wasi	GHC WASM backend (GHC 9.6+)
Grain	wasm32	grain compiler (WASM-first language)
Python	wasm32-wasi	CPython's WASI build (Tier 3), or Cython → WASI SDK
Ruby	wasm32-wasi	ruby.wasm (mruby/wrb)

All application languages compile to the same ABI: WASI Preview 1 for standard I/O (files, clocks, random numbers) and SixOS native host functions for kernel-specific operations (networking, graphics, input, IPC, windowing). Anything that can produce a WASI Preview 1 module — today or in the future — runs on SixOS without any OS-side changes.

3.2 System Services (Native Rust)

The system boots with a chain of Rust services, each responsible for a subsystem. They are compiled with the kernel and shipped in the OS image — they are not WASM modules and need no sandbox, because they are part of the TCB (trusted computing base).

3.2.1 init — Service Manager

The first service started by the kernel. It brings up all system services in dependency order: core services first (networking, graphics, input), then the window manager, then the desktop shell, and finally launches user applications. If a service crashes, init restarts it.

3.2.2 netd — Networking Service

Provides TCP/IP networking over VirtIO network devices. Uses the smoltcp networking stack for IPv4, TCP, UDP, and DNS resolution. Manages socket creation, connection establishment, data transfer, and name resolution. Applications communicate with netd through IPC channels — they don't access network hardware directly.

3.2.3 gpud — Graphics Service

Manages the display system through the VirtIO GPU device. Handles display initialization, framebuffer allocation, scanout configuration, and resource management. Implements a WebGPU-native API for GPU compute and rendering, including shader compilation, resource binding, command encoding, and surface presentation. Applications use WebGPU pipelines for accelerated graphics and compute workloads. When GPU acceleration is unavailable or unsupported, gpud also provides a software rendering fallback, allowing applications to run through the same API with reduced performance but without requiring code changes.

3.2.4 hidd — Input Service

Collects input events from VirtIO keyboard and mouse devices. Translates key codes to ASCII characters (handling shift and caps modifiers). Routes keyboard input and mouse motion to the active application through the window manager.

3.2.5 appmgrd — Application Manager

The central windowing and application lifecycle service. Manages windows (creation, destruction, positioning, focus), composites the desktop framebuffer from individual window contents, routes input events from hidd to the correct application, and coordinates with gpud for display output.

3.2.6 shelld — Desktop Shell

The graphical desktop environment. Provides a full-screen background with a taskbar, application launcher, system tray, and window decorations. Draws

title bars with close buttons, handles window dragging, and displays running applications.

3.3 The Browser Application

The flagship userspace application is a web browser written in Go and compiled to WASM. It demonstrates the full capabilities of the SixOS platform:

- **Windowed display:** Creates a window through the app manager, receives keyboard and mouse events, presents rendered frames to the compositor.
- **HTTPS client:** Full HTTP/1.1 GET and HTTPS requests over TCP/TLS sockets managed by the networking stack. DNS resolution via the kernel's DNS host function.
- **HTML parser:** Uses the `golang.org/x/net/html` library to parse HTML documents, extract title, text content, and hyperlinks.

The browser runs as a standard WASM process with no special privileges. It accesses the network, input, and display exclusively through host functions — the same interface available to any application.

3.4 Filesystem

SixOS provides multiple filesystem layers:

- **ramfs:** An in-memory filesystem used for the root during boot. Fast, no persistence, automatically populated with system binaries at startup.
- **VirtIO Block:** Persistent storage through QEMU's VirtIO block device. Provides sector-level read/write access to a disk image.
- **Btrfs:** A copy-on-write filesystem with CRC32C data integrity verification, subvolumes, and snapshot support. Designed as the primary on-disk filesystem, with checksums on every metadata and data block.

The VFS (Virtual Filesystem) layer provides a unified file API through which all filesystem implementations are accessed. The file descriptor table maps open files to per-process handles with capability-gated access.

3.5 Two-Layer POSIX Compatibility

SixOS provides POSIX functionality at two levels:

Layer 1 — Sandboxed (all applications): The WASI Preview 1 standard interface covers files, clocks, random number generation, and command-line I/O. This is available to all WASM applications without special privileges. It provides roughly 70% POSIX coverage — enough for most application code.

Layer 2 — Privileged (selected WASM workloads): WASM modules that hold the SYSP0SIX capability get full POSIX support including processes (fork, exec, wait), signals, memory mapping (mmap/munmap), pipes, epoll, and filesystem operations (mount, chroot). This enables porting complex POSIX-dependent software while keeping it capability-gated — the OS core itself needs no such layer, being native Rust with direct kernel access.

Applications without the SYSP0SIX capability simply cannot invoke these operations — the kernel enforces this at the host function dispatch level.

3.6 The Application Model

A SixOS application is a .wasm module that calls host functions. The standard pattern:

1. Connect to the app manager service via IPC channel
2. Create a window (specify title, dimensions, position)
3. Enter a main loop:
 - Poll for events (key presses, mouse clicks, window resize)
 - Render the application’s framebuffer
 - Present the framebuffer to the compositor
 - Yield to the scheduler
4. On window close, clean up and exit

No special runtime, no dynamic linker, no system libraries to link against. The WASM module contains everything it needs, and the host functions provide the rest.

4 SixOS — Supported Devices Plan

SixOS targets RISC-V 64-bit silicon. The device plan below is ordered by proximity to the current development platform (QEMU virt) — each tier reuses the same kernel, driver interface, and WASM application ABI.

4.1 Tier 1 — Reference Development Platform (Current)

QEMU virt machine. Full VirtIO device set: block, network, GPU, input (keyboard/mouse), RNG. This is where all kernel and userspace development happens today. Everything in this tier already works.

4.2 Tier 2 — Single-Board Computers

RISC-V dev boards that mirror the QEMU virt device model closely:

- **StarFive VisionFive 2** — quad-core JH7110 (RV64GC), VirtIO-capable via PCIe, HDMI output. Primary candidate for first hardware bring-up.
- **Milk-V Mars / Mars CM** — JH7110-based, compute-module form factor.
- **SiFive HiFive Unmatched** — Freedom U740, the most mature upstream Linux RISC-V board; useful for low-level validation.

Goal in this tier: boot SixOS natively, drive a display, and run the full WASM desktop stack (app manager, shell, browser) on real hardware.

4.3 Tier 3 — Handheld

Battery-powered, touch-first devices. These drive the roadmap for power management, touch input, and small-screen UI:

- **Tablet** — touch display with a multi-window shell layout. Stylus input via the input daemon. Target hardware class: RISC-V SoC application processors (e.g., quad-core RV64GC).
- **Smart gadgets** (watches, e-readers, portable media devices) — constrained variants of the handheld stack: read-mostly UI, aggressive suspend/resume, and a reduced set of system services.

Key OS work for this tier: runtime power management, thermal policy, touch and gesture input pipeline, display panels (DSI/eDP), and a mobile-oriented desktop shell.

4.4 Tier 4 — IoT and Edge

Headless or minimally-interactive devices where SixOS's determinism and capability security are the selling points:

- **Smart home controllers** — sensors, actuators, Matter/Thread bridges.
- **Industrial gateways** — protocol conversion, real-time control loops benefiting from the $< 50 \mu\text{s}$ interrupt latency guarantee.
- **Network appliances** — routers and firewalls built on the smoltcp networking stack.

These devices typically run a trimmed OS image: kernel, netd, and a handful of WASM application modules, with no graphics stack. The capability model gives each device function a minimal, auditable authority surface.

4.5 Tier 5 — UAV and Aerial Robotics

Unmanned aerial vehicles split across the two sides of the device plan: a hard real-time flight controller and a full-featured companion computer, both running the same kernel.

Flight controller (SixOS-RT). The control side runs the no-WASM variant so the image contains only the kernel and statically linked native Rust tasks — statically analyzable worst-case execution time for attitude, motor, and failsafe loops. Target hardware class: RISC-V MCU-class SoCs with hardware timers, CAN/serial ESC buses, and PWM or DSHOT motor outputs. Requirements shaping this tier:

- **Determinism** — sub-50 μ s interrupt latency, core pinning, and the priority ceiling protocol for the control loop, with telemetry and logging on separate best-effort priorities that cannot preempt it.
- **Capability-gated actuators** — arming, motor output, and geofence authority are tokens. A payload or plugin that was never granted the motor capability cannot arm the craft, no matter what code it runs.
- **Failsafe in the TCB** — return-to-home, geofence enforcement, and link-loss handling are trusted native tasks, not sandboxed apps.

Companion computer (full SixOS). The application side runs the web-native variant:

- **Vision and mapping payloads** — camera pipelines and on-device inference for obstacle avoidance, landing-pad detection, and inspection or mapping analysis, accelerated through the WebGPU-native graphics service.
- **Telemetry and link** — flight telemetry, RTSP/video streaming, and MAVLink-style messaging over the smoltcp networking stack.
- **Ground station and mission apps** — mission planning, live video, and log review are ordinary SixOS web applications addressed by URL, with over-the-air updates delivered through the platform’s normal update path.

Because both sides share one kernel, one driver interface, and one capability model, the companion computer’s apps are sandboxed and least-privilege while the control loop stays fully deterministic — the split a UAV architecture needs, without maintaining two operating systems.

4.6 Tier 6 — Automotive IVI

In-vehicle infotainment head units. Requirements that shape this tier:

- **Functional-safety-aware architecture** — the trusted computing base (kernel + core services in Rust) kept small and auditable, with applications isolated in WASM sandboxes. A crash in a media app cannot take down the instrument cluster.
- **Determinism** — hard real-time scheduling guarantees for safety-relevant displays and input paths.
- **Rich UI** — multi-display (center stack, cluster, HUD), hardware accelerated rendering via the WebGPU-native graphics service.

- **Connectivity** — networking and an over-the-air update pipeline, all running as capability-gated WASM applications.

4.7 What Each Tier Shares

One kernel, one driver interface, one WASM application ABI. A browser .wasm built for QEMU virt runs unchanged on a tablet or an IVI head unit — only the device tree and the set of active system services differ.

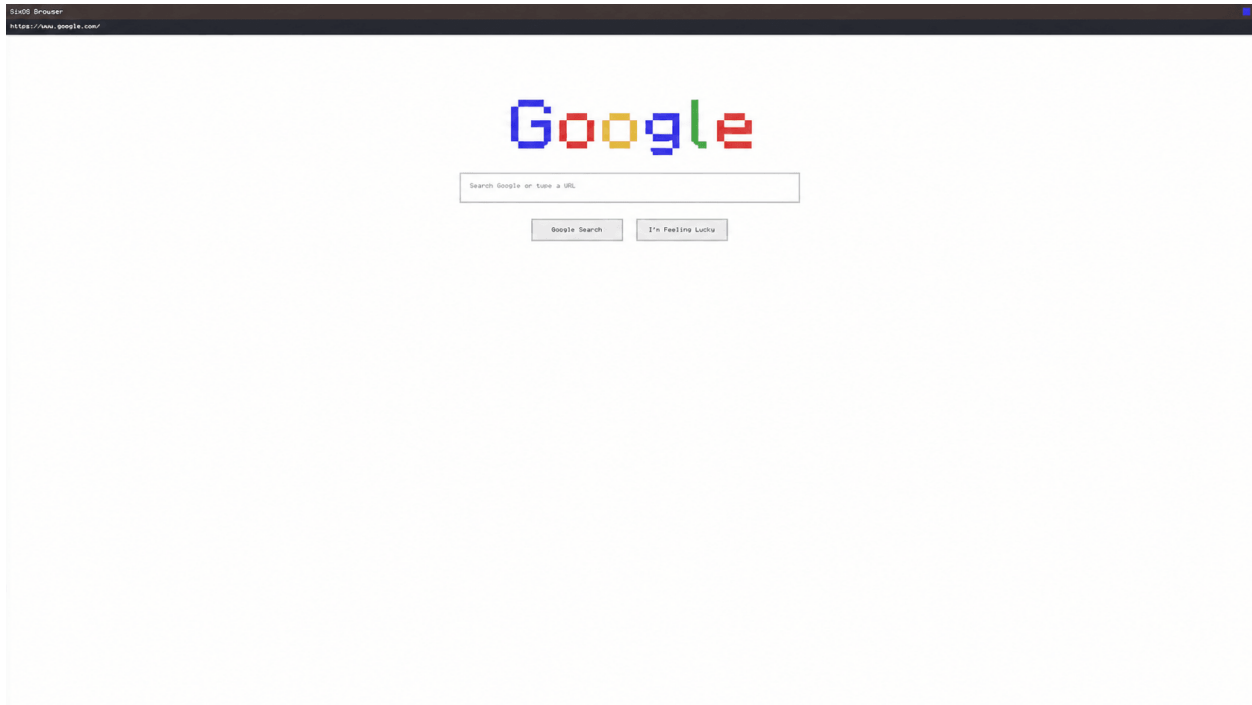
5 SixOS — Current Status

As of September 2026.

SixOS is a working system, not a proposal. The kernel boots, the WASM runtime executes real applications, the desktop stack composites windows, and the flagship browser fetches and renders pages over the network — all on QEMU virt today, with physical hardware bring-up underway.

```
[SixOS] VirtIO MMIO @ 0x10001000: magic=0x74728976 devid=0
[SixOS] VirtIO MMIO @ 0x10002000: magic=0x74728976 devid=0
[SixOS] VirtIO MMIO @ 0x10003000: magic=0x74728976 devid=0
[SixOS] VirtIO MMIO @ 0x10004000: magic=0x74728976 devid=0
[SixOS] VirtIO MMIO @ 0x10005000: magic=0x74728976 devid=0
[SixOS] VirtIO MMIO @ 0x10006000: magic=0x74728976 devid=0
[SixOS] VirtIO MMIO @ 0x10007000: magic=0x74728976 devid=0
[SixOS] VirtIO MMIO @ 0x10008000: magic=0x74728976 devid=0
[SixOS] ERROR: No virtio-net device found, using 0x10002000
[SixOS] Using virtio-net at 0x10002000
[VIO magic=74728976 ver=1 devid=0 vid=554D4551 feat=0 mmax=0]
  drvfeat=0[QSET q=0 pfn=81E96 rb=0 mmax=0 desc=81E96000]
[QSET q=1 pfn=81E98 rb=0 mmax=0 desc=81E98000]
[SixOS] MAC: {00000
[VIO status=0]
[VIO net_status=0]
[VIO irq_status=0]
[SixOS] Network initialized
[SixOS] MAC: 00:00:00:00:00:00
[SixOS] Window manager initialized
[SixOS] Running browser via wasmi runtime...
[SixOS] Calling _start...
Attempting to fetch https://google.com...
HTTPS connecting: google.com:443
[TCP_CONNECT addr=65 78 61 6D 70 6C 65 2E 63 6F 6D 3A 34 34 33 ]
[DNS_QUERY google.com]
[TX desc_addr=85F9A710 desc=85F9A710:34:0:0 avail_idx=0 ring=0 idx=1 flags=0]
[TX consumed=0 used_idx=0 tx_idx=0 total=34 data_len=2A hdr=A bytes=0 0 0 0 0 0 0 0 0 0 FF
FF FF FF FF 0 0 0 0 0 8 6 0 1 8 0 6 4 0 1 0 0 0 0 0 0 A 0 2 F FF FF FF FF FF FF ]
```

SixOS booting on QEMU virt: VirtIO device scan, network initialization, window manager startup, and the WASM browser fetching <https://google.com> over TCP/TLS with DNS resolution



The WASM browser rendering Google homepage in a composited window

5.1 Kernel — Working

- **Boot:** OpenSBI → kernel entry on RV64, identity-mapped address space, Devicetree-driven hardware discovery.
- **Memory management:** Physical page allocator, kernel heap, per-process WASM linear-memory management.
- **Scheduler:** Priority-preemptive multi-core scheduling (up to 8 HARTs), fixed-priority and EDF classes, priority ceiling protocol, per-CPU run-queues with work stealing.
- **IPC:** Capability-gated channels with bounded ring buffers, blocking and non-blocking operations, notifications, shared-memory transfer.
- **Security:** Full capability system — mint, derive, transfer, revoke — enforced at the host-function dispatch level.
- **Drivers:** UART, PLIC, VirtIO block / net / GPU / input / RNG.

5.2 WASM Runtime — Working

- Bytecode interpreter executing the full WASM instruction set — no JIT
- WASI Preview 1 host functions: files, clocks, random, stdio.
- SixOS native host functions: networking, windowing, input, IPC.
- Conformance coverage tracked via the official spec test suite.
- POSIX compatibility layer with two tiers: sandboxed WASI for all apps, capability-gated SYSPOSIX for privileged workloads.

5.3 System Services — Working

The full service chain boots in dependency order and is restarted by init on crash:

- **init** — service manager and boot orchestration
- **netd** — TCP/UDP/IPv4 + DNS over smoltcp, DHCP client
- **gpud** — VirtIO GPU display, framebuffer management, WebGPU-native API with software-rendering fallback
- **hidd** — keyboard/mouse collection, key translation, input routing
- **appmgrd** — window lifecycle, compositing, focus, input dispatch
- **shelld** — desktop shell with taskbar, launcher, window decorations
- **perfd / traced** — performance and tracing support daemons

5.4 Applications — Working

- **Browser** (Go → WASM): the flagship app. Windowed display through the app manager, HTTPS client over the system network stack, HTML parsing via golang.org/x/net/html, page rendering with hyperlinks.

5.5 Hardware Bring-Up — In Progress

The **Orange Pi RV2** port (8-core Ky X1 SoC, HDMI, dual GbE, NVMe) is documented: boot chain, DTB parsing, PLIC fixes, framebuffer and serial bring-up, Ethernet, and 8-HART SMP bootstrap — tracked in a dependency-ordered master checklist.

6 SixOS — The Real-Time Variant

SixOS’s kernel is not tied to WebAssembly. The same kernel — scheduler, memory management, capabilities, IPC, drivers — can be shipped in a second configuration with **no WASM runtime at all**: a hard real-time OS (RTOS) for deeply embedded targets where every microsecond and every kilobyte counts.

6.1 Why a No-WASM Variant

The WASM sandbox is the right trade-off for general-purpose devices: it buys security and portability at the cost of interpretation overhead. But that cost is unacceptable in two situations:

- **Hard real-time loops** — a motor controller or safety interlock cannot spend cycles decoding bytecode. Native code with statically known worst case execution time is the only honest answer.

- **Deeply constrained targets** — a sensor node with 64 KB of RAM cannot host a bytecode interpreter and per-module linear memories.

For these targets, SixOS ships as a pure RTOS: same kernel, native Rust applications, no interpreter in the image.

6.2 What Stays the Same

The variant is a build-time configuration of one codebase, not a fork:

- **The kernel is identical** — scheduler, memory management, capability system, IPC, notifications, drivers. One code base, one set of tests, one set of bug fixes. A driver fixed for the RTOS variant is fixed for the desktop variant the same day.
- **Capability security stays.** Capabilities are plain tokens, not a WASM artifact. Native RTOS tasks hold capabilities exactly as WASM apps do — least-privilege authority without the sandbox tax.
- **The API surface stays.** Host-function semantics (channels, files, devices) map one-to-one onto RTOS system calls. Code written against the SixOS API moves between variants by recompiling, not rewriting.
- **The real-time guarantees stay** — and now apply to *all* code on the system, not just the kernel: sub-50 μ s interrupt latency, priority ceiling protocol, core pinning, per-CPU runqueues.

6.3 What Changes

- **No WASM runtime in the image** — the interpreter, module loader, and linear-memory manager are compiled out. The image contains only the kernel and statically linked native Rust tasks.
- **Applications are native Rust tasks**, built with the kernel image and flashed as one firmware. No dynamic loading — deployment is firmware update, not app installation.
- **Trust model inverts.** WASM apps are untrusted and sandboxed; RTOS tasks are trusted code compiled into the TCB. The safety guarantee comes from the Rust type system and the small, auditable codebase — not from a sandbox.
- **IPC is the composition boundary.** Without a shared sandbox ABI, tasks interact through the same capability-gated channels, so the system is still a set of communicating components rather than one monolithic firmware blob.
- **No POSIX compatibility layer.** The two-tier POSIX layer (sandboxed WASI + privileged SYSPOSIX) exists to host ported legacy software — and there is no legacy software on an RTOS target. The variant ships the native SixOS API only: channels, notifications, files, devices. Nothing to

emulate means nothing to audit, nothing to misconfigure, and no compatibility surface an attacker can lean on.

6.4 Where Each Variant Runs

	SixOS (full)	SixOS-RT (no WASM)
Primary use	Desktop, tablet, IVI, UAV companion computers	Robotics, IoT nodes, UAV flight control, industrial control
Applications	WASM modules (any language)	Native Rust tasks, statically linked
Isolation	WASM sandbox + capabilities	Capabilities + Rust correctness
POSIX layer	WASI + capability-gated SYSPOIX	None — native API only
Memory	MBs-GBs	64 KB-MBs
Latency budget	Interactive + soft real-time	Hard real-time loops
Distribution	URLs, on-device install	Firmware image

The device plan maps naturally: desktops, tablets, IVI head units, and UAV companion computers run the full variant with its web-native app ecosystem; robotics controllers, IoT nodes, UAV flight controllers, and industrial control units run the RT variant. A UAV is the clearest example of both sides at once: SixOS-RT holds the control loop on the flight controller while the full variant runs the companion computer’s payload and ground-station apps. More generally, a gateway that bridges them — SixOS-RT on the control side, full SixOS on the management side — is the same kernel running twice.

6.5 Why This Matters

Most operating systems force a choice: a rich, safe application platform, or a small deterministic one. Because SixOS’s safety story is built from capabilities (which need no sandbox) plus Rust (which needs no runtime), the kernel can be both. The WASM runtime and the POSIX layer become features you include — not assumptions you are stuck with.

7 Appendix A — Capability-Based Security in Context

Why SixOS chose capabilities, and how that choice compares to the alternatives.

7.1 The Question

Every operating system answers the same question: **when a program wants to do something, how does the system decide whether to allow it?** The answer shapes the attack surface, what a bug can destroy, and whether an AI agent can safely act on a user's behalf. SixOS answers with capabilities. Here is that answer against the four mainstream alternatives.

7.2 The Alternatives

Unix permissions (ACLs). Authority follows *user identity*: every process can touch anything its user can. Simple, universal — and structurally broken. The exploited text editor can read your SSH keys because it runs as *you* (ambient authority). Authority follows the program's identity, not the caller's intent, which enables the classic confused-deputy attack. Every mitigation (sudo, containers, seccomp) is a patch on this, not a fix.

Monolith + hardening. The kernel holds all authority; security means keeping userspace from tricking it. Hardening — seccomp, namespaces, pledge/unveil — shrinks the *probability* of a bug reaching the payload, but not the *target*: a 30-million-line C kernel with total authority, parsing attacker input at every syscall. A permanent, pragmatic arms race.

Microkernel isolation. Push drivers and services into separate address spaces; failures are contained, and seL4 adds formal verification. The strongest competitor — and itself capability-based; its access control *is* capabilities. The costs are the IPC tax per cross-space call and complexity that moves rather than vanishes.

Language sandbox (Java, V8, WASM). Safety enforced in software, not hardware — near-zero isolation cost. The right mechanism for containing untrusted code, but it answers only “can this code hurt the kernel?” WASI's capability-oriented turn shows the field converging on what it still lacks.

Capabilities. Authority is not who you are but what you hold: an unforgeable token for a specific object with specific rights. No token, no access. Tokens mint, narrow, transfer, revoke. Honest costs: capability management is a design burden, no global namespace is a UX problem, and capability-native software ecosystems are small.

7.3 Capability-Based Systems

Capabilities are not a SixOS invention — they are a decades-old idea with a track record. The lineage and what SixOS takes from each:

- **KeyKOS / EROS / Coyotos** (1970s–2000s). The pioneers: persistent capability machines where a single running system’s entire state was reachable only through tokens. Proved the model works at OS scale. SixOS inherits the core vocabulary (mint, derive, transfer, revoke).
- **seL4** (2006). Capabilities as the *only* access control, plus a formally verified kernel — mathematical proof that the mechanism is correct. SixOS takes the cap-system design; declines the microkernel IPC tax by keeping capabilities in-kernel.
- **Capsicum** (FreeBSD, 2010s). Capabilities retrofitted onto Unix: processes start ambient, then drop into “capability mode.” Proved the model in production — but as a layer *under* an unchanged Unix, not a replacement. SixOS is what Capsicum points at when freed from compatibility debt.
- **Fuchsia / Zircon** (Google, 2016). The most commercially backed capability OS: handles (tokens) are the only way to reach kernel objects; no root, no global file namespace. Validates the model at phone scale.
- **Genode** (2008). Capability-based component framework where each component sees only its own resources; parents delegate to children. Closest in spirit to SixOS’s service tree, with a heavier focus on nested virtualization.
- **WASI** (2019). Not an OS, but the WASM system interface, whose newer APIs are explicitly capability-oriented (resources passed as handles). SixOS’s bet: the web platform is converging on the same answer from the application side.

The pattern across four decades: capabilities win wherever a system is designed without legacy constraints — and get bolted on where it isn’t. SixOS simply starts where the winners started.

7.4 Worked Example: The Same Afternoon, Under Five Models

A user opens a malicious photo in an image editor and asks the assistant to “resize this and upload it to my album.” The decoder bug succeeds under every model. Only the blast radius differs.

- **Unix**: the attacker *is* the user — SSH keys, documents, cookies, all reachable. Mitigation: hope the container config was right.
- **Monolith**: same outcome, reached by defeating each hardening layer; one kernel bug restores full authority.
- **Microkernel**: the crash is contained — but a *live* compromised process

still reaches whatever capabilities it accumulated, e.g. a whole-directory grant.

- **WASM sandbox:** the kernel is safe; the engine never asks whether the code may touch the user’s files.
- **Capabilities (SixOS):** the editor’s cap space is exactly

```
[cap 12] file  /inbox/photo_7741.jpg    READ
[cap 13] ipc   → window manager        SEND
(nothing else – no home cap, no network cap)
```

The exploited process can read the photo it already had and draw windows. `~/.ssh/id_rsa` has no path to it — there is no user identity for authority to ride on. The assistant spawns an agent with a narrowed manifest set — file READ, album SEND, AI session with a token budget — so “resize and upload” works and “email all my documents to the attacker” fails for want of tokens. Closing the photo revokes the session, recursively killing every derived token.

One bug, five afternoons. Only the last ends with the attacker holding what they already had. Sandboxes and capabilities answer different questions — “can it hurt the kernel?” vs. “can it hurt the user?” — and SixOS needs both answers.

7.5 Side-by-Side

Property	Unix ACLs	Monolith + hardening	Microkernel	Language sandbox	Capabilities
Authority follows	user identity	kernel state	server + caps	runtime config	held tokens
Ambient authority	yes (structural)	yes	partial	partial	none
Confused-deputy safe	no	no	partial	partial	yes
Isolation cost per call	syscall	syscall	IPC round trip	~zero	~zero (in-kernel)
Least-privilege granularity	per-user	per-process	per-capability	per-engine	per-object, per-right
TCB size	kernel + setuid	entire kernel	verified μ -kernel	engine + kernel	kernel + core services
AI-agent-ready	no	bolt-on	yes	no (alone)	yes (native)

7.6 Why This Decision Is Load-Bearing

SixOS's other choices *depend* on capabilities, not merely coexist with them: WASM-only userspace works because tokens replace the page table as the security boundary; AI-first works because an agent is just a process with tokens; no POSIX layer is viable because no legacy debt forces ambient authority back in; the RT variant inherits the same token model for native tasks. Remove capabilities and every one of those sentences needs a different answer — the definition of a load-bearing design decision.